

Experiment 2

Problem Statement:

Data Collection - Capture social media data from platforms (e.g., Twitter, Facebook, LinkedIn, YouTube) using APIs such as Twitter API v2, Facebook Graph API, and YouTube Data API, along with scraping and crawling techniques (e.g., BeautifulSoup, Selenium)

Theory:

Social Media Data Analysis – The process of collecting raw social media data and measuring it against your goals and objectives.

Data Collection – It is the process of collecting information from relevant sources to find a solution to the given statistical inquiry. Collection of Data is the first and foremost step in a statistical investigation. It's an essential step because it helps us make informed decisions, spot trends, and measure progress.

In the context of social media, data collection involves extracting posts, comments, likes, shares, hashtags, videos, and other engagement metrics. This data can be used for various applications, including market research, social sentiment analysis, and recommendation systems.

There are two primary ways to collect data from social media platforms:

1. **APIs (Application Programming Interfaces):** Most social media platforms provide APIs that allow developers to access and retrieve structured data programmatically.
2. **Web Scraping & Crawling:** When APIs do not provide sufficient access, web scraping tools like BeautifulSoup and Selenium are used to extract data directly from web pages.

Methods of Data Collection –

Both the methods are discussed in detail as follows:

1. **APIs for Social Media Data Retrieval:**
 - **Twitter API v2:** Provides access to tweets, user engagement metrics, and historical data.
 - **Facebook Graph API:** Allows developers to collect posts, comments, reactions, and page insights.
 - **LinkedIn API:** Used for extracting professional profiles, posts, and network connections.
 - **YouTube Data API:** Enables access to video metadata, comments, and analytics.
2. **Web Scraping and Crawling Techniques:**
 - **BeautifulSoup:** A Python library used for parsing HTML and XML documents, allowing efficient extraction of web content.
 - **Selenium:** A browser automation tool that can simulate user actions, making it useful for scraping dynamic content.

Overview of Y-Combinator –

Y Combinator is a well-known startup accelerator that helps early-stage companies grow by providing seed funding, mentorship, and networking opportunities. It has backed some of the most successful startups, including Airbnb, Dropbox, Stripe, and Reddit. Y Combinator provides a structured program where startups refine their ideas, develop their business models, and prepare for investor pitches. In the context of data collection and analysis, many startups leverage Y Combinator's support to build data-driven applications and platforms

By performing social media analysis on Y Combinator, we aim to –

1. **Identify Trends –**

Track the most discussed topics related to Y Combinator, including startup funding, new technologies, and entrepreneurial success stories.

2. **Sentiment Analysis –**

Analyze the general sentiment of discussions about Y Combinator, its alumni startups, and its investment decisions.

3. **Engagement Metrics –**

Measure social media interactions such as likes, shares, and comments on posts related to Y Combinator.

4. **Influencer Detection –**

Identify key individuals and organizations that drive discussions about Y Combinator on platforms like Twitter and LinkedIn.

5. **Startup Success Insights –**

Track how Y Combinator-backed startups are perceived and discussed on social media over time.

Web Scraping –

Web scraping is an automatic method to obtain large amounts of data from websites. Most of this data is unstructured data in an HTML format which is then converted into structured data in a spreadsheet or a database so that it can be used in various applications. There are many different ways to perform web scraping to obtain data from websites. These include using online services, particular API's or even creating your code for web scraping from scratch. Many large websites, like Google, Twitter, Facebook, StackOverflow, etc. have API's that allow you to access their data in a structured format.

Web scraping requires two parts, namely **the crawler** and **the scraper**. The crawler is an artificial intelligence algorithm that browses the web to search for the particular data required by following the links across the internet. The scraper, on the other hand, is a specific tool created to extract data from the website.

How Web Scrapers Work –

Web Scrapers can extract all the data on particular sites or the specific data that a user wants. Ideally, it's best if you specify the data you want so that the web scraper only extracts that data quickly. For example, you might want to scrape an Amazon page for the types of juicers available, but you might only want the data about the models of different juicers and not the customer reviews.

When a web scraper needs to scrape a site, first the URLs are provided. Then it loads all the HTML code for those sites and a more advanced scraper might even extract all the CSS and Javascript elements as well. Then the scraper obtains the required data from this HTML code and outputs this data in the format specified by the user. Mostly, this is in the form of an Excel spreadsheet or a CSV file, but the data can also be saved in other formats, such as a JSON file.

Types of Web Scrapers –

Web Scrapers can be divided on the basis of many different criteria, including Self-built or Pre-built Web Scrapers, Browser extension or Software Web Scrapers, and Cloud or Local Web Scrapers.

Self-built Web Scraper requires advanced knowledge of programming. And if you want more features in your Web Scraper, then you need even more knowledge. On the other hand, pre-built Web Scrapers are previously created scrapers that you can download and run easily. These also have more advanced options that you can customize.

Browser extensions Web Scrapers are extensions that can be added to your browser. These are easy to run as they are integrated with your browser, but at the same time, they are also limited because of this. Any advanced features that are outside the scope of your browser are impossible to run on Browser extension Web Scrapers. But Software Web Scrapers don't have these limitations as they can be downloaded and installed on your computer. These are more complex than Browser web scrapers, but they also have advanced features that are not limited by the scope of your browser.

Cloud Web Scrapers run on the cloud, which is an off-site server mostly provided by the company that you buy the scraper from. These allow your computer to focus on other tasks as the computer resources are not required to scrape data from websites. Local Web Scrapers, on the other hand, run on your computer using local resources. So, if the Web scrapers require more CPU or RAM, then your computer will become slow and not be able to perform other tasks.

BeautifulSoup –

BeautifulSoup is a Python library for web scraping and parsing HTML and XML documents, giving us more options to navigate through a structured data tree. The library can parse and navigate through the page, allowing you to extract information from the HTML or XML code by providing a simple, easy-to-use API.

Advantages –

- It's faster.
- BeautifulSoup's beginner-friendly and easier to set up.
- The library works independently from browsers.
- It requires less time to run.
- It can parse HTML and XML documents.
- The framework's easier to debug.

Disadvantages –

- It can't interact with web pages like a human user.
- The library can just parse data. Therefore, you'll need to install other modules to extract the information, e.g., requests or httpx.
- It solely supports Python.
- You'll need a different module to scrape JavaScript-rendered pages since BeautifulSoup only lets you navigate through HTML or XML files.

Significance of BeautifulSoup –

BeautifulSoup is best used for web scraping tasks that involve parsing and extracting information from static HTML pages and XML documents. For instance, if you need to retrieve data from a website with a simple structure, such as a blog or an online store, the Python library can easily extract the information you need by parsing the HTML code.

However, if you're looking to scrape dynamic content, Selenium is a better alternative.

Selenium –

Selenium is an open-source browser automation tool often used for web scraping. Here are its main components:

- **Selenium IDE** – used to record actions before automating them.
- **Selenium WebDriver** – a web automation tool that empowers you to control web browsers.
- **Selenium Grid** – helpful for parallel execution.

Advantages –

- It's easy to use.
- The library supports multiple programming languages, like JavaScript, Ruby, Python, and C#.
- It can automate Firefox, Edge, Safari, and even a custom QtWebKit browser.
- Selenium can interact with the web page's JavaScript code, execute XHR requests, and wait for elements to load before scraping the data. In other words, you can scrape dynamic web pages handily. It also has anti-bot bypass plugins like the Undetected ChromeDriver. All these increase its chances of evading blocks during web scraping.

Disadvantages –

- Selenium set-up methods are complex.
- It uses more resources compared to BeautifulSoup.
- It can become slow when you start scaling up your application.

Significance of Selenium –

Selenium is ideal for scraping websites that require interaction with the page, e.g., filling out forms, clicking buttons, or navigating between pages. For instance, if you need to retrieve information from a login-protected site, Selenium can automate the login process and navigate through the pages to get the data.

Comparison between BeautifulSoup & Selenium –

Criterion	Beautiful Soup	Selenium
Ease of use	User-friendly	Complex to set up and use
Programming languages	Only Python	Python, C#, JavaScript, PHP, Java and Perl
Browser support	It doesn't require a browser instance	Chrome, Brave, Edge, IE, Safari, Firefox and Opera
Performance	Faster since it just requires the page source	Slower as it uses a web driver instance to navigate through the web
Functionality	Mainly used for parsing and extracting information from HTML and XML documents	Selenium can automate browser actions, like clicking buttons, filling out forms and navigating between pages
Operating system support	Windows, Linux and macOS	Windows, Linux and macOS
Architecture	Simple HTTP and XML parser to navigate through structured data	JSON Wire protocol to manage Web Drivers
Prerequisites	BeautifulSoup package and a module to send requests (like as httpx)	Selenium Bindings and Browser Drivers
Dynamic content	It only works with static web pages	Selenium can scrape dynamically generated content